Phạm Mạnh Thắng – B22DCVT527

Architecture and API Documentation - BookStore Microservices

1. Introduction

1.1 Background

Traditional monolithic systems package all modules into one deployable artifact and often use one shared database. While this model can be easier at project start, growth introduces technical and organizational friction. In a BookStore context, identity, catalog, cart, order, payment, shipping, review, and recommendation evolve at different speeds and have different runtime profiles.

1.2 Problem Statement

A monolith for BookStore causes:

* Hard scaling for specific hot paths (for example catalog browsing).

* Risky deployment because all modules are released together.

* Tight coupling across teams and code modules.

* Technology lock-in and lower parallel development throughput.

1.3 Objective

The objective is to implement an academic yet realistic microservice architecture that:

* Decomposes domain capabilities into independent services.

* Enforces database-per-service boundaries.

* Exposes API-first REST contracts.

* Satisfies assignment functional requirements.

* Runs with Docker Compose and supports service-level testing.

2. From Monolith to Microservices

2.1 Monolithic Baseline

Monolith characteristics relevant to BookStore:

* Single codebase, single runtime process.

* Shared database schema.

* In-process module calls.

* One deployment pipeline.

Advantages at small scale:

* Simple local setup.

* Easier transaction handling across modules.

* One place to debug initially.

2.2 Monolith Limitations

As scope increases, practical issues appear:

* Scaling mismatch: checkout traffic and catalog traffic differ; monolith scales all or nothing.

* Release risk: a change in review module can impact checkout deployment.

* Team contention: merge conflicts and long integration cycles.

* Schema coupling: changing one bounded context may break another query path.

2.3 Microservice Transition Rationale

Microservices are selected to isolate business capabilities and operational concerns. The BookStore domain naturally fits service decomposition because each context (identity, catalog, ordering, payment, shipping, review, recommendation) can be modeled with explicit API contracts and independent persistence.

3. Design Principles Applied

3.1 Single Responsibility at Service Level

Each service owns one business capability:

* `customer-service` handles customer records.

* `cart-service` handles carts and cart items.

* `pay-service` handles payment records.

This reduces cognitive load and allows focused testing.

3.2 Database per Service

No service directly reads another service database. Data sharing is performed by API calls and business identifiers (`customer_id`, `book_id`, `order_id`). This supports autonomy and schema evolution.

3.3 API-First Design

The system exposes explicit REST endpoints for each service. Current version focuses on `GET` and `POST` contracts required by assignment scenarios.

3.4 Loose Coupling

Dependencies are explicit and narrow:

* `customer-service` -> `cart-service` (cart auto creation)

* `cart-service` -> `book-service` (book validation)

* `gateway` -> downstream services for UI rendering

3.5 Stateless Services

Service endpoints are stateless in request processing (no in-memory session dependency). Authentication with JWT is listed as principle; current academic implementation can add JWT in next iteration.

3.6 Resilience

Current code includes basic request timeout in selected calls. Full resilience stack (retry, circuit breaker, fallback policy) is proposed in Section 10 as improvement roadmap.

4. DDD Decomposition

4.1 Bounded Context Mapping

The domain decomposition follows assignment guidance:

* Identity context: `customer-service`, `staff-service`, `manager-service`

* Catalog context: `catalog-service`, `book-service`

* Ordering context: `cart-service`, `order-service`

* Payment context: `pay-service`

* Shipping context: `ship-service`

* Review context: `comment-rate-service`

* Recommendation context: `recommender-ai-service`

* Access context: `api-gateway`

4.2 Ubiquitous Language

Core domain terms used consistently in APIs and models:

* Customer, Staff, Manager

* Catalog, Book

* Cart, CartItem, Order

* Payment, Shipment

* CommentRate, Recommendation

4.3 Context Mapping

Integration style by context:

* Upstream identity to ordering: customer creation triggers cart creation.

* Upstream catalog to ordering: cart validates book before insertion.

* Ordering to payment/shipping: order references downstream operational records.

4.4 Aggregate and Data Ownership

Each service controls its aggregate root and persistence:

* `Cart` aggregate owns `CartItem` child collection.

* `Order` aggregate remains isolated from payment/shipment storage.

* `Recommendation` aggregate stores model outputs independent of transactional tables.

5. Target Architecture Overview

5.1 System Topology

* External clients connect to API Gateway.

* API Gateway exposes user-facing routes and forwards/aggregates downstream data.

* 11 domain services handle business APIs.

* Each service has private persistence.

5.2 Communication Pattern

Primary pattern:

* Synchronous REST over HTTP.

Logical flows represented in diagrams:

* `cart-service` validates book existence via `book-service`.

* `order-service` has business dependency to payment and shipping.

* recommendation and review services use cross-context identifiers.

5.3 Deployment Pattern

* Containerized per service with Dockerfile.

* Orchestrated via docker-compose.

* Enabls independent restart and scaling per service.

5.4 Architecture Quality Attributes

* Scalability: selective scale-out by service role.

* Maintainability: bounded codebases per domain.

* Reliability: service failure impact localized.

* Evolvability: API contract boundaries support iterative changes.

6. Service Implementation Summary

6.1 API Gateway

Responsibilities:

* Single entry point for UI templates and service navigation.

* Proxy and aggregate calls to downstream APIs.

* Health endpoint for quick liveness check.

Notable routes:

* `/books/`, `/customers/`, `/cart-items/`, `/cart-detail/`

* `/services/` and `/services/{service_key}/`

6.2 Identity Services

`customer-service`

* Model: `Customer(name, email unique)`

* API: `GET/POST /customers/`

* Business side effect: `POST /carts/` at cart-service after customer creation.

`staff-service`

* Model: `Staff(name, email unique, role)`

* API: `GET/POST /staffs/`

`manager-service`

* Model: `Manager(name, email unique)`

* API: `GET/POST /managers/`

6.3 Catalog Services

`catalog-service`

* Model: `Catalog(name, description)`

* API: `GET/POST /catalogs/`

`book-service`

* Model: `Book(title, author, price, stock)`

* API: `GET/POST /books/`

6.4 Ordering Services

`cart-service`

* Models: `Cart(customer_id)`, `CartItem(cart, book_id, quantity)`

* APIs:

* `GET/POST /carts/`

* `GET /carts/{id}/`

* `POST /cart-items/`

* Validation logic: Queries `book-service /books/` and rejects unknown `book_id`.

`order-service`

* Model: `Order(customer_id, status)`

* API: `GET/POST /orders/`

6.5 Payment and Shipping

`pay-service`

* Model: `Payment(order_id, amount, status)`

* API: `GET/POST /payments/`

`ship-service`

* Model: `Shipment(order_id, address, status)`

* API: `GET/POST /shipments/`

6.6 Review and Recommendation

`comment-rate-service`

* Model: `CommentRate(customer_id, book_id, comment, rating)`

* API: `GET/POST /comment-rates/`

`recommender-ai-service`

* Model: `Recommendation(customer_id, book_id, score)`

* API: `GET/POST /recommendations/`

7. Assignment Functional Requirement Coverage

FR1. Customer registration automatically creates a cart Implemented in `customer-service` `POST /customers/`:

* Save customer.

* Call `cart-service POST /carts/` with `customer_id`.

Result: A customer record and corresponding cart are created in separate services.

FR2. Staff manages books Current implementation provides service-level CRUD foundation:

* `book-service POST /books/` creates books.

* Gateway exposes `/books/` page for manual operation.

Note: Role-based authorization (staff only) can be added via JWT and policy middleware.

FR3. Customer adds books to cart, views cart, updates cart Covered partially and strongly for add/view:

* Add item: `cart-service POST /cart-items/`

* View cart: `cart-service GET /carts/{id}/`

* Book existence validated via book-service call.

Gap: Explicit update endpoint for cart item quantity is not yet implemented; proposed in roadmap.

FR4. Order triggers payment and shipping Current APIs exist for order/payment/shipping services.

* `order-service` manages order entities.

* `pay-service` and `ship-service` manage operational records.

Gap: Automatic orchestration from order create to payment/shipment creation is currently represented as logical flow in diagram; implementation can be extended with orchestration or event pattern.

FR5. Customer can rate books Implemented:

* `comment-rate-service POST /comment-rates/`

* Stores rating and comment with customer/book references.

8. API Documentation Strategy

Comprehensive API contracts are provided in `API_DOCUMENTATION.md`:

* Endpoint listing by service.

* Request/response examples.

* Status codes and error semantics.

* Quick cURL commands for validation.

Recommended extension for final submission:

* Export OpenAPI/Swagger per service.

* Add gateway-level unified API index.

9. Deployment, Build, and Execution

9.1 Containerization

Each service includes:

* Independent `Dockerfile`

* `requirements.txt`

* Own Django project and app

9.2 Orchestration

`docker-compose.yml` coordinates service startup and networking.

Expected benefits:

* Consistent environment.

* Repeatable demo flow.

* Simple service restart and observation.

9.3 Independent Scaling Example

Examples:

* Scale catalog during browse spikes.

* Scale cart/order during checkout campaigns.

Command pattern (conceptual):

* `docker compose up --scale catalog-service=3`

10. Reliability and Resilience Assessment

10.1 Current State

Observed resilience elements:

* Timeout in selected HTTP requests (`timeout=5` in gateway and customer/cart service calls).

10.2 Gaps

* No retry policy for transient downstream failures.

* No circuit breaker to prevent cascading failures.

* No message broker for async decoupling in critical workflows.

* No idempotency key for repeated client submissions.

10.3 Improvement Plan

Priority-ordered plan:

1. Add retries with exponential backoff for safe idempotent GET requests.

2. Introduce circuit breaker around inter-service calls.

3. Add request timeout and fallback responses consistently across all services.

4. Introduce event bus (RabbitMQ/Kafka) for order-created workflows.

5. Add distributed tracing (OpenTelemetry) and correlation IDs.

11. Security Considerations

11.1 Current State

* Basic service exposure for academic environment.

* No unified auth token enforcement yet.

11.2 Recommended Security Controls

* JWT-based auth at gateway and service validation.

* Role-based authorization for staff/manager operations.

* Input validation hardening for all payload fields.

* Rate limiting and abuse protection on gateway routes.

* Secrets management for credentials and service configs.

12. Testing and Verification

12.1 Functional Tests Executed

Core flows verified:

* Customer creation and auto-cart side effect.

* Book creation and listing.

* Cart item insertion with book validation.

* Cart detail retrieval.

* Gateway route availability and service list rendering.

12.2 Suggested Automated Test Matrix

* Unit tests for serializer/model validation per service.

* API integration tests for inter-service calls.

* Contract tests between gateway and service endpoints.

* Negative tests for invalid IDs and malformed payloads.

* Resilience tests for timeout and downstream unavailability.

12.3 Acceptance Criteria Mapping

* Diagram per service: completed and explained.

* API documentation: completed.

* Technical report: this document.

* Remaining deliverables (repo and demo video): operational tasks outside this report text.

13. Discussion

13.1 Benefits Achieved

* Clear modular decomposition and bounded contexts.

* Service-level autonomy and data ownership.

* Better extensibility for future features (auth, events, analytics).

13.2 Trade-offs

* Increased operational complexity compared to monolith.

* More network calls and potential latency.

* Need stronger observability and resilience patterns for production-readiness.

13.3 Academic vs Production Gap

This implementation intentionally prioritizes architecture structure and assignment required flows. For production, additional non-functional controls are mandatory (SLOs, security hardening, async workflows, incident tooling).

14. Conclusion

The BookStore system has been successfully decomposed into microservices aligned with assignment objectives and DDD boundaries. The architecture demonstrates independent service ownership, database-per-service isolation, and REST-based collaboration with API Gateway as the controlled entry point. Required business flows are implemented for customer registration with cart auto-creation, cart-book validation, and rating flow, while order-payment-

shipping orchestration is represented and can be incrementally upgraded to event-driven execution.

Overall, the project provides a valid academic microservice foundation and a clear roadmap toward production-grade reliability, security, and observability.

15. Detailed Scenario Walkthroughs

This section describes end-to-end behavior for the most important flows. It can be used as speaking script for demo video and as evidence for architecture correctness.

15.1 Scenario A - Customer Registration with Automatic Cart Creation

Preconditions: `customer-service` and `cart-service` are healthy.

Steps:

1. Client submits `POST /customers/` with `name` and `email`.

2. `customer-service` validates payload and inserts new customer row.

3. `customer-service` sends `POST /carts/` to `cart-service` with `{ customer_id }`.

4. `cart-service` creates a cart owned by this customer.

5. `customer-service` returns `201` to caller.

Postconditions:

* New customer exists in `customer-db`.

* New cart exists in `cart-db` with matching `customer_id`.

Failure notes:

* In current implementation, cart call result is not wrapped in compensation logic.

* Improvement option: transactional outbox + event-driven cart creation.

15.2 Scenario B - Add Book to Cart with Book Validation

Preconditions: Book exists in `book-service`. Cart exists in `cart-service`.

Steps:

1. Client submits `POST /cart-items/` with `cart`, `book_id`, `quantity`.

2. `cart-service` calls `GET /books/` on `book-service`.

3. `cart-service` checks whether `book_id` exists in response list.

4. If valid, `cart-service` saves `CartItem`; otherwise returns `404 Book not found`.

Postconditions: Cart item is stored only when reference integrity (logical) is satisfied.

Failure notes:

* `GET /books/` list scan is simple but not optimal at scale.

* Improvement option: `GET /books/{id}` endpoint and caching.

15.3 Scenario C - Order, Payment, and Shipment Lifecycle

Current behavior in code base:

* `order-service`, `pay-service`, and `ship-service` are implemented as separate APIs.

* End-to-end orchestration is currently represented at architecture level and can be manually exercised through API sequence.

Suggested orchestrated flow (next iteration):

1. Create order (`status=pending`).

2. Trigger payment creation.

3. If payment succeeds, create shipment.

4. Update order status to `processing` then `completed`.

Recommended design option: Use async domain events (`OrderCreated`, `PaymentSucceeded`, `ShipmentCreated`) to reduce runtime coupling.

15.4 Scenario D - Rating and Recommendation

Steps:

1. Customer posts review to `comment-rate-service`.

2. Recommendation service periodically reads interaction features and writes scores to `recommendation` table.

3. Gateway exposes recommendation list for UI consumption.

Discussion:

* This preserves separation between transactional ordering and analytical recommendation.

* Recommendation model evolution does not require checkout service deployment.

16. Architecture Decision Records (ADR Summary)

ADR-01: Use API Gateway as Single Public Entry - Decision: External clients access only gateway routes.

* Reasoning: Simplifies client integration and internal endpoint evolution. Enables future centralized concerns: auth, throttling, logging.

* Consequences: Gateway becomes critical component and must be monitored.

ADR-02: Database per Service - Decision: Every service owns private persistence.

* Reasoning: Avoids tight schema coupling and cross-team migration conflicts.

* Consequences: Cross-service joins are impossible at DB level and must be composed via APIs/events.

ADR-03: REST for Initial Integration - Decision: Use synchronous REST for first academic iteration.

* Reasoning: Simpler implementation and debugging for coursework timeline.

* Consequences: Higher runtime coupling than event-driven approach. Need resilience controls for production use.

ADR-04: Keep Services Stateless - Decision: No server-side user session required by service logic.

* Reasoning: Better horizontal scaling and simpler failover.

* Consequences: Authentication context should move to JWT token in future iteration.

17. Performance and Scalability Considerations

17.1 Read-heavy vs Write-heavy Paths

* Read-heavy: catalog, books, recommendation queries.

* Write-heavy: cart updates during campaigns, order and payment during checkout peaks.

Scaling guidance: Prefer scaling catalog/book independently from payment/shipping. Add caching for hot read endpoints.

17.2 Latency Hotspots

Potential hotspots:

* Gateway fan-out views that query many services.

* `cart-service` validation call to `book-service`.

Mitigation:

* Add timeout, retry for idempotent reads, and bounded concurrency in gateway.

* Introduce service-level metrics (p95 latency, error rate).

17.3 Data Growth

High-growth entities: `CartItem`, `CommentRate`, `Recommendation` records as model refresh frequency increases.

Mitigation: Add retention and archival policy. Partitioning strategy for analytics-heavy tables.

18. Roadmap (Iteration Plan)

Iteration 1 - Contract and Security Hardening - Add JWT authentication and role checks.

* Publish OpenAPI docs per service.

* Add request validation constraints and standardized error schema.

Iteration 2 - Reliability Upgrade - Introduce retry/circuit breaker library.

* Add centralized logging and distributed tracing.

* Implement health/readiness endpoints in all services.

Iteration 3 - Event-driven Order Workflow - Introduce message broker.

* Emit and consume domain events for payment and shipping.

* Add idempotency keys to prevent duplicate operations.

Iteration 4 - Productization - CI/CD pipeline with service-level tests.

* Container image scanning and dependency checks.

* SLO dashboards and alerting policy.

Appendix A - Service List

1. `api-gateway`

2. `customer-service`

3. `staff-service`

4. `manager-service`

5. `catalog-service`

6. `book-service`

7. `cart-service`

8. `order-service`

9. `pay-service`

10. `ship-service`

11. `comment-rate-service`

12. `recommender-ai-service`

API Documentation - Bookstore Microservices

Base setup:

* API Gateway: `http://localhost:8000`

* Internal service ports are mapped by Docker Compose.

* Current implementation mostly provides list/create APIs (`GET`, `POST`).

1. API Gateway

GET / - Description: Render home page.

* Response: HTML page.

GET|POST /books/ - Description: View and create books via gateway UI.

* POST payload:

```
{
  "title": "Clean Architecture",
  "author": "Robert C. Martin",
  "price": "19.99",
  "stock": 20 }
```

GET|POST /customers/ - Description: View and create customers via gateway UI.

* POST payload:

```
{
  "name": "Phạm Mạnh Thắng",
```

"email": "phammanhthang@example.com" }

* Side effect: `customer-service` auto-calls `cart-service` to create cart.

GET|POST /cart-items/ - Description: Add item to a cart via gateway UI.

* POST payload:

{

"cart": 1,

"book_id": 2,

"quantity": 1 }

GET|POST /cart-detail/ - Description: Enter `cart_id` and view cart details.

GET /services/ - Description: Show links to all registered downstream service list pages.

GET /services/{service_key}/ - Description: Generic list page for one downstream service.

* Supported keys: `order`, `pay`, `ship`, `comment-rate`, `staff`, `manager`, `catalog`, `recommender`.

GET /health/ - Description: Gateway health probe.

* Response body: `ok`.

2. customer-service

Base path (inside cluster): `http://customer-service:8000`

GET /customers/ - Description: Get all customers.

* Response item:

{

"id": 1,

"name": "Phạm Mạnh Thắng",

"email": "phammanhthang@example.com" }

POST /customers/ - Description: Create customer.

* Request:

{

"name": "Phạm Mạnh Thắng",

"email": "phammanhthang@example.com" }

* Behavior: after save, service calls `POST http://cart-service:8000/carts/` with `{ "customer_id": 1 }`.

* Success: `201 Created`.

* Validation: unique email.

3. staff-service

Base path: `http://staff-service:8000`

GET /staffs/ - Description: List staff records.

POST /staffs/ - Description: Create staff record.

* Request:

{

"name": "Staff A",

"email": "staff@example.com",

"role": "inventory" }

* Validation: unique email.

4. manager-service

Base path: `http://manager-service:8000`

GET /managers/ - Description: List managers.

POST /managers/ - Description: Create manager.

* Request:

{

"name": "Manager A",

"email": "manager@example.com" }

* Validation: unique email.

5. catalog-service

Base path: `http://catalog-service:8000`

GET /catalogs/ - Description: List catalogs.

POST /catalogs/ - Description: Create catalog.

* Request:

{

"name": "Software Engineering",

"description": "Books about architecture and design" }

6. book-service

Base path: `http://book-service:8000`

GET /books/ - Description: List books.

POST /books/ - Description: Create a book.

* Request:

{

"title": "Clean Architecture",

"author": "Robert C. Martin",

"price": "19.99",

"stock": 20 }

* Field types: `price`: decimal, `stock`: integer.

7. cart-service

Base path: `http://cart-service:8000`

GET /carts/ - Description: List carts.

POST /carts/ - Description: Create a cart.

* Request:

{

"customer_id": 1 }

GET /carts/{cart_id}/ - Description: Get one cart with items.

* Response:

{

  "cart": {

    "id": 1,

    "customer_id": 1   },

  "items": [

    {

      "id": 10,

      "cart": 1,

      "book_id": 2,

      "quantity": 1   }   ] }

POST /cart-items/ - Description: Add item into cart.

* Request:

{

  "cart": 1,

  "book_id": 2,

  "quantity": 1

}

* Validation behavior: Calls `GET http://book-service:8000/books/`. Returns `404` if `book_id` not found.

8. order-service

Base path: `http://order-service:8000`

GET /orders/ - Description: List orders.

POST /orders/ - Description: Create order.

* Request:

```
{
  "customer_id": 1,
  "status": "pending"
}
```

9. pay-service

Base path: `http://pay-service:8000`

GET /payments/ - Description: List payments.

POST /payments/ - Description: Create payment.

* Request:

```
{
  "order_id": 1,
  "amount": "99.50",
  "status": "pending"
}
```

10. ship-service

Base path: `http://ship-service:8000`

GET /shipments/ - Description: List shipments.

POST /shipments/ - Description: Create shipment.

* Request:

```
{
  "order_id": 1,
  "address": "Ha Noi",
  "status": "preparing"
}
```

11. comment-rate-service

Base path: `http://comment-rate-service:8000`

GET /comment-rates/ - Description: List comments and ratings.

POST /comment-rates/ - Description: Create a comment/rating.

* Request:

```
{
  "customer_id": 1,
  "book_id": 2,
  "comment": "Good book",
  "rating": 5
}
```

12. recommender-ai-service

Base path: `http://recommender-ai-service:8000`

GET /recommendations/ - Description: List recommendations.

POST /recommendations/ - Description: Save recommendation score.

* Request:

```
{
  "customer_id": 1,
  "book_id": 2,
  "score": 0.93
}
```

Common HTTP status codes

* `200 OK`: successful GET.

* `201 Created`: successful POST.

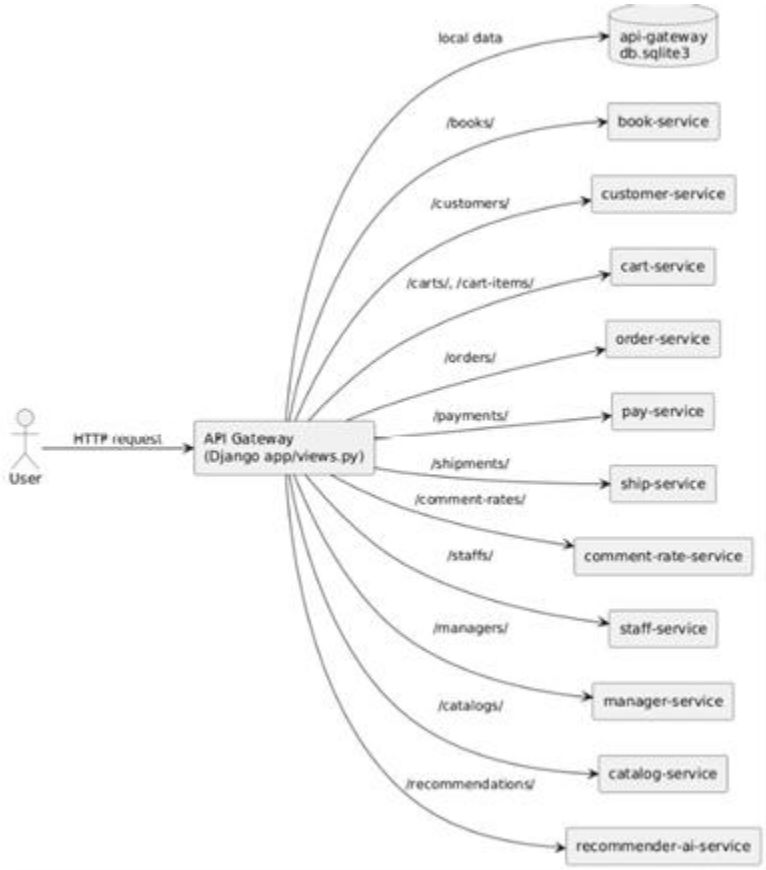
* `400 Bad Request`: serializer validation error or missing required fields.

* `404 Not Found`: resource not found (for example invalid `book_id` in cart-item).

* `5xx`: downstream connectivity/runtime error.

Architecture Diagram

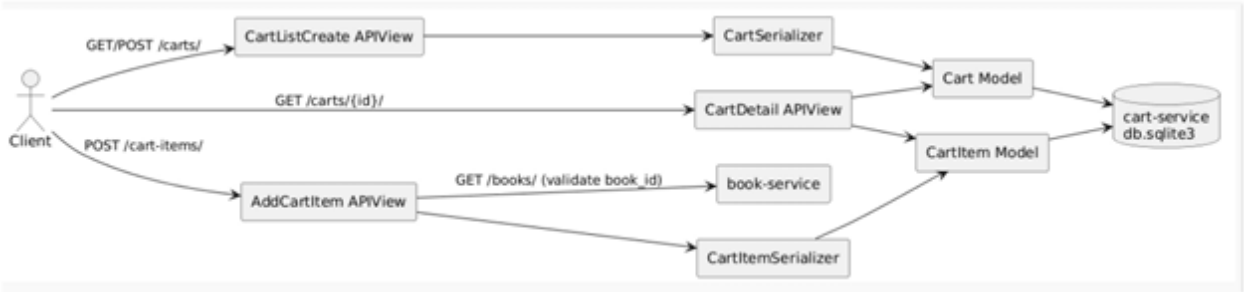
1. API GATEWAY Architecture



2. Customer Architecture

3. Book Architecture

4. Cart Architecture



5. Order Architecture

6. Payment Architecture

7. Shipment Architecture

8. Comment Architecture

9. Staff service Architecture

10. Manager Architecture

11. Recomenda on Architecture